

ARTIFICIAL INTELLIGENCE

Lecture 00: Search

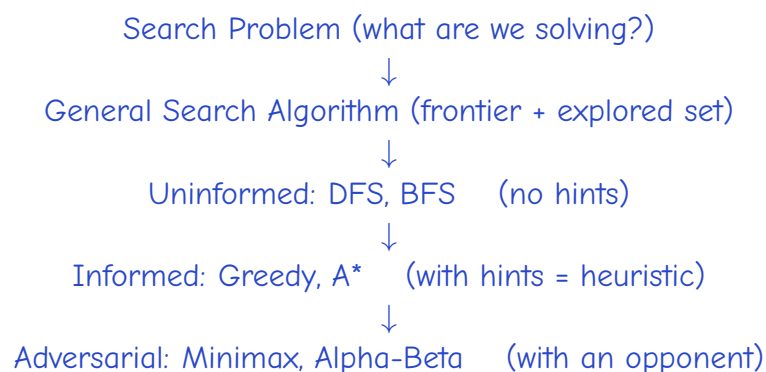
Aifee Aadil

(Based on Harvard CS50's Introduction to AI — theory & concepts)

What this lecture teaches:

1. What AI is, and where "Search" fits in.
2. How to describe any problem as a **search problem** (the 5 parts).
3. The general search algorithm: nodes, frontier, explored set.
4. **Uninformed search**: Depth-First Search (DFS) and Breadth-First Search (BFS).
5. **Informed search**: heuristics, Greedy Best-First Search, and A* Search.
6. **Adversarial search** (games): Minimax, Alpha-Beta Pruning, Depth-Limited Minimax.

Big Picture of this Lecture



Basic Words You Need First (Zero Knowledge Needed)

Read this page first. Every hard word in these notes is built from these simple ideas.

Word	Simple meaning
Algorithm	A step-by-step recipe to solve a problem. Like a cooking recipe: do step 1, then step 2 ... until done.
Problem	Something we want the computer to figure out (e.g. “what is the shortest way home?”).
Goal	The thing we want to reach (the exit of a maze, a solved puzzle, winning a game).
Graph	A drawing of dots connected by lines . Dots = places/situations. Lines = ways to move between them. Like a metro map.
Data structure	A way of organising information inside the computer (a list, a pile, a line).
List	Items written one after another: [A, B, C].
Function	A small machine: you give it input, it gives you output. e.g. <code>ADD(2, 3)</code> gives 5. In these notes, <code>ACTIONS(s)</code> means “give me the moves possible in situation <i>s</i> ”.
Cost	How “expensive” something is: distance, time, money, or number of steps.
Optimal	The best possible (usually the cheapest / shortest).
Estimate	A guess based on some information (“it’s about 5 km from here”).
Recursion	A function that uses itself to solve a smaller version of the problem. Like Russian dolls: open one, there’s a smaller one inside, until the smallest.
Infinity (∞)	A number bigger than any number. $-\infty$ is smaller than any number. Used as a “starting worst value”.

Think of it like this (real life):

You are lost in a new city and want to reach a restaurant.

You are standing somewhere (**start**). You can walk down different streets (**actions**). Every street takes you to a new corner (**new situation**). You want to reach the restaurant (**goal**) walking as little as possible (**lowest cost**).

This whole lecture is about teaching a computer to do exactly this — in a smart way.

1. What is Artificial Intelligence?

Artificial Intelligence (AI) = any technique that lets a computer behave in a way that looks **intelligent** — solving problems, making decisions, learning from data, and understanding language or images.

AI is not one single thing. It is a **collection of techniques**. The CS50 AI course covers 7 of them:

Topic	What it means	Real-life example
0. Search	Find a sequence of moves from a start to a goal.	Google Maps route, solving a puzzle, a chess AI.
1. Knowledge	Store facts and use logic to find new facts. ($P \rightarrow Q$, P , so Q)	"If it rains, the ground is wet. It rains. So the ground is wet."
2. Uncertainty	Make decisions using probability when we are not 100% sure.	Weather forecast: 70% chance of rain.
3. Optimization	Choose the best option from many possible options.	Making an exam timetable with no clashes.
4. Learning	Improve by learning from data instead of fixed rules.	Email: Inbox vs Spam.
5. Neural Networks	Models inspired by the human brain.	Face recognition, handwriting reading.
6. Language	Understand and produce human language (NLP).	ChatGPT, Google Translate.

Simple:

AI = many techniques that make computers act smart. **Lecture 0 = Search:** finding a way from a start to a goal.

2. Search Problems

Search problem = a problem where an agent starts in some situation and must find a **sequence of actions** that takes it to a **goal** situation.

Example:

- **15-puzzle** → slide tiles on a 4×4 board until they are in order 1–15.
- **Maze** → find the way from the start square (A) to the exit square (B).
- **Driving directions** → find a route from home to university.

All three look different, but they are the **same kind of problem**. So one algorithm can solve all of them.

Think of it like this (real life):

Think of a **board game**. There is a starting position, rules about which moves you may make, each move changes the board, and there is a winning position. A "search problem" is just a precise way of writing down these 4–5 things so a computer can play.

The Vocabulary of Search

1. Agent

Agent = an entity that **perceives** (senses) its environment and **acts** upon that environment.

Examples: the program solving the puzzle, a robot vacuum, a self-driving car, a chess program.

Environment → **Agent senses it** → **Agent chooses action** → **Environment changes**

2. State

State = a **configuration** of the agent and its environment — a complete “snapshot” of the situation at one moment.

Example:

- 15-puzzle → one particular arrangement of the tiles.
- Maze → which square you are standing on.
- Chess → the position of every piece + whose turn it is.

3. Initial State

Initial state = the state in which the agent **begins**.

Example: the shuffled 15-puzzle you are given; square A in the maze.

4. Actions

Actions = the choices that can be made in a state.

We write it as a function:

ACTIONS(s) → returns the set of actions that can be executed in state s

Example:

15-puzzle: the empty space can be filled by sliding a neighbouring tile.

If the empty space is in a **corner** → only **2** actions possible.

If the empty space is in the **middle** → **4** actions possible (up, down, left, right).

So **ACTIONS(s)** **depends on the state**. Different states allow different actions.

5. Transition Model

Transition model = a description of **what state results** from performing any applicable action in any state.

RESULT(s, a) → returns the state you get after doing action a in state s

Example:

s = puzzle where tile 12 is next to the empty space.

a = “slide 12 into the empty space”.

$\text{RESULT}(s, a)$ = the same puzzle, but 12 has moved and the empty space is where 12 was.

Why?

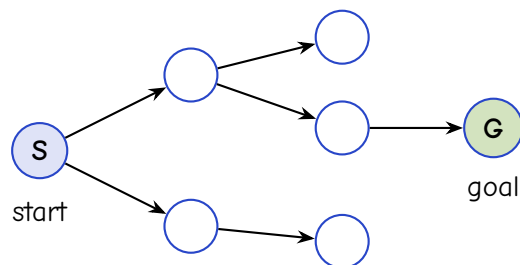
The computer cannot “see” the puzzle. RESULT tells it what will happen **before** actually doing it, so it can plan ahead.

6. State Space

State space = the set of **all states reachable** from the initial state by any sequence of actions.

We draw the state space as a **graph**:

- each **circle (vertex)** = a state,
- each **arrow (edge)** = an action that moves from one state to another.



The 15-puzzle has over **10 trillion** reachable states, so we can never draw it all — the algorithm has to explore it piece by piece.

7. Goal Test

Goal test = a way to determine whether a given state is a **goal state**.

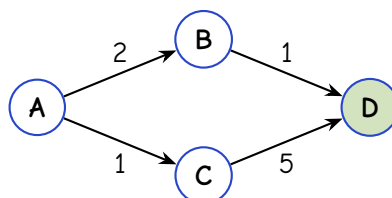
Example:

- 15-puzzle → “Are the tiles in order 1, 2, ..., 15?”
- Maze → “Am I on square B?”
- Sometimes there are **many** goal states (e.g. “any checkmate position” in chess).

8. Path Cost

Path cost = a numerical cost associated with a given path (sequence of actions).

Each action has a **step cost**. Path cost = sum of all step costs on the path.



- Path $A \rightarrow B \rightarrow D \rightarrow$ cost $2 + 1 = 3$.
- Path $A \rightarrow C \rightarrow D \rightarrow$ cost $1 + 5 = 6$.

Both reach D, but $A \rightarrow B \rightarrow D$ is cheaper.

In many problems (maze, 15-puzzle) every step costs the **same** (cost = 1), so path cost = number of steps.

Solution and Optimal Solution

Solution = a sequence of actions that leads from the initial state to a goal state.

Optimal solution = a solution that has the **lowest path cost** among all solutions.

Example:

In the graph above: both $A \rightarrow B \rightarrow D$ and $A \rightarrow C \rightarrow D$ are **solutions**.

Only $A \rightarrow B \rightarrow D$ (cost 3) is the **optimal** solution.

There can be **many** solutions but the optimal one is the **cheapest**. If two solutions have the same lowest cost, both are optimal.

All Parts Together – Three Examples

Part	15-Puzzle	Maze	Driving Directions
State	Arrangement of tiles	Current square	Current location (city/junction)
Initial state	Shuffled board	Square A	Home
Actions	Slide a tile into the gap	Move up/down/left/right (not into walls)	Take a road from here
Transition model	Board with that tile moved	The neighbouring square	The town at the end of that road
Goal test	Tiles in order 1-15?	On square B?	At university?
Path cost	1 per move	1 per step	Distance (km) or time

Easy Memory Trick:

A search problem has 5 parts: **I**nitial state, **A**ctions, **T**ransition model, **G**oal test, **P**ath cost
→ “**I** Always Take **G**ood **P**aths”

3. Nodes — How the Computer Remembers a Path

Think of it like this (real life):

Hansel and Gretel dropped **breadcrumbs** while walking into the forest, so they could find their way back. A node is a note that says: “I am *here*, I came *from there*, by doing *this move*, and it cost me *this much* so far.”

Node = a data structure that keeps track of: a **state**, a **parent**, an **action**, and a **path cost**.

Field	Meaning
state	Where we are now.
parent	The node that generated this node (where we came from).
action	The action applied to the parent to get to this node.
path cost	Total cost from the initial state to this node.

State vs Node: a *state* is just the situation. A *node* is the state **plus** its history (how we got there). The same state can appear in two different nodes if we reached it by two different paths.

Why do we store the parent?

When we finally reach the goal, we only have the **goal node**. To know the full path, we follow the **parent** links backwards:

E (parent C) → C (parent B) → B (parent A) → A (no parent = start)

Collected backwards: E, C, B, A. Reverse it → **A → B → C → E**.

Simple:

A node = a state + a “breadcrumb” back to where it came from.

4. The General Search Algorithm

Think of it like this (real life):

You are searching a shopping mall for a shoe shop.

You carry a notebook with two lists:

- “**Places to check**” → shops you have seen but not entered yet. (= **frontier**)
- “**Already checked**” → shops you already went inside. (= **explored set**)

Each time: pick a shop from “places to check”, go in. Is it the shoe shop? If yes, done! If not, write it in “already checked”, and add any *new* shops you can see from there to “places to check”. If “places to check” becomes empty and you never found it → there is no shoe shop.

Frontier = the collection of nodes that we have **discovered but not yet explored**. It is the “to-do list” of the search.

Expand a node = look at all the actions possible in that node’s state, and create the resulting neighbour nodes.

Basic Approach

1. Start with a frontier that contains the **initial state**.
2. Repeat:
 - (a) If the frontier is **empty** → there is **no solution**. Stop.
 - (b) **Remove** a node from the frontier.
 - (c) If the node contains a **goal state** → return the solution. Stop.
 - (d) **Expand** the node, and add the resulting nodes to the frontier.

What could go wrong? – Infinite Loops

Suppose roads go **both ways**: $A \rightarrow B$ and $B \rightarrow A$.



Remove A → expand → add B
 ↓
 Remove B → expand → add A **again**
 ↓
 Remove A → expand → add B **again**
 ↓
 ... forever — the algorithm never finishes!

Revised Approach — with an Explored Set

Explored set = the set of states we have **already explored**. We never add them to the frontier again.

1. Start with a frontier that contains the initial state.
2. Start with an **empty explored set**.
3. Repeat:
 - (a) If the frontier is empty → no solution.
 - (b) Remove a node from the frontier.
 - (c) If the node contains a goal state → return the solution.
 - (d) **Add the node to the explored set**.

- (e) Expand the node, and add resulting nodes to the frontier **only if they are not already in the frontier or the explored set**.

Simple:

Keep a list of places you've already been, and never go back to them.

Remember

The whole algorithm stays the same. The **only** thing that changes is **which node we remove** from the frontier:

- Remove the **newest** node → **DFS** (use a stack).
- Remove the **oldest** node → **BFS** (use a queue).
- Remove the node with the **smallest** $h(n)$ → **Greedy Best-First**.
- Remove the node with the **smallest** $g(n) + h(n)$ → **A***.

5. Stack and Queue

1. Stack — Last-In First-Out (LIFO)

Stack = a data type where the **last** item added is the **first** item removed.

Example:

A pile of plates: you put a plate on **top**, and you take a plate from the **top**.

add 1, add 2, add 3 → remove gives 3, then 2, then 1

2. Queue — First-In First-Out (FIFO)

Queue = a data type where the **first** item added is the **first** item removed.

Example:

A line at the bank: the person who came first is served first.

add 1, add 2, add 3 → remove gives 1, then 2, then 3

Easy Memory Trick:

Stack → Same end in & out (top)
line)

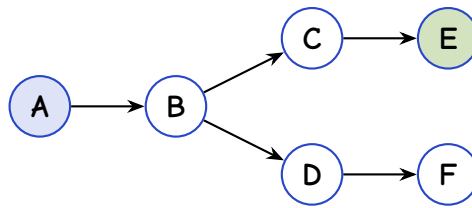
Queue → in at the back, out at the front (a **Queue** line)

6. Uninformed Search

Uninformed search = a search strategy that uses **no problem-specific knowledge**. It does not know which direction leads closer to the goal.

It explores “blindly” in a fixed order. DFS and BFS are both uninformed.

We will trace both on this graph. **Task: find a path from A to E.**



(When B is expanded, C is added first, then D.)

1. Depth-First Search (DFS)

Think of it like this (real life):

Exploring a cave. You take the first tunnel and keep walking **as far as it goes**. When you hit a dead end, you walk back to the last place where the tunnel split, and try the other way. You always finish one tunnel completely before trying another.

Depth-First Search = a search algorithm that always expands the **deepest** node in the frontier.

The frontier is a **stack**. Because the newest node is always taken next, DFS keeps going **deeper and deeper** along one path. Only when it hits a **dead end** does it go back (**backtrack**) and try the next option.

DFS trace (A to E)

Step	Remove	Frontier after	Explored	What happened
0	–	[A]	{}	Start with A in the frontier.
1	A	[B]	{A}	A not goal. Add B.
2	B	[C, D]	{A, B}	B not goal. Add C, then D.
3	D	[C, F]	{A, B, D}	D is on top (last in). Add F.
4	F	[C]	{A, B, D, F}	F has no neighbours → dead end.
5	C	[E]	{A, B, D, F, C}	Backtrack to C. Add E.
6	E	–	–	E is the goal! Stop.

Path (follow parents from E): **A → B → C → E.**

DFS explored 6 nodes. It **wasted** time on D and F because it went deep in the wrong direction first.

DFS in a maze

DFS picks one corridor and runs down it to the very end. If it's a dead end, it goes back to the last junction and tries another corridor.

- If it picks the right corridor early → very **fast**, explores little.
- If it picks wrong → may explore almost the **whole maze**.
- The path it finds may be **long and winding**, even if a short path exists.

Properties of DFS

- **Complete?** (always finds a solution if one exists) → Yes for a **finite** state space (with an explored set). No for an **infinite** one — it can go down forever.
- **Optimal?** → **No**. It returns the first path it finds, not the shortest.
- **Memory** → usually **low**: it only needs to remember the current path and its siblings.

Simple:

DFS = go as deep as possible, then backtrack. Frontier = Stack. Fast if lucky, not always shortest.

2. Breadth-First Search (BFS)

Think of it like this (real life):

Looking for someone who knows a doctor. First you ask **all your friends**. If none of them know one, you ask **all the friends of your friends**. Then friends of friends of friends. You finish one whole “circle” before moving to the next one. So the first person you find is the **closest** connection to you.

Breadth-First Search = a search algorithm that always expands the **shallowest** node in the frontier.

The frontier is a **queue**. Because the oldest node is taken next, BFS explores **level by level**: first all nodes 1 step away, then all nodes 2 steps away, then 3 ...



It spreads out like **ripples in water** from the start.

BFS trace (A to E)

Step	Remove	Frontier after	Explored	What happened
0	–	[A]	{}	Start with A.
1	A	[B]	{A}	Add B (level 1).
2	B	[C, D]	{A, B}	Add C, D (level 2).
3	C	[D, E]	{A, B, C}	C is at the front (first in). Add E.
4	D	[E, F]	{A, B, C, D}	Finish level 2. Add F.
5	E	–	–	E is the goal! Stop.

Path: **A** → **B** → **C** → **E**. BFS finished in 5 steps and never explored F.

BFS in a maze

BFS explores all squares 1 step from A, then all squares 2 steps from A, and so on. It spreads out evenly in every direction until it touches B.

- Always finds the **shortest** path.
- But often explores **many more squares** than needed (including ones far from the goal).

Properties of BFS

- **Complete?** → **Yes**, if the number of actions per state is finite.
- **Optimal?** → **Yes**, when every step has the **same cost** (it finds the path with the fewest steps).
- **Memory** → **high**: it has to store the whole current level, which can be huge.

Simple:

BFS = explore level by level. Frontier = Queue. Always shortest (equal costs), uses lots of memory.

DFS vs BFS — Full Comparison

Point	DFS	BFS
Expands	Deepest node	Shallowest node
Frontier	Stack (LIFO)	Queue (FIFO)
Search shape	One long path at a time	Level by level (ripples)
Complete?	Only in finite spaces	Yes
Optimal?	No	Yes (equal step costs)
Memory	Low	High
Best when	Many solutions, deep goals, little memory	Need the shortest path, goal is near the start

Common Mistake:

Saying “DFS is always faster than BFS”. Not true. DFS can be much faster *or* much slower — it depends on luck (which branch it tries first). BFS is predictable but uses more memory.

Easy Memory Trick:

DFS → **D**eep → **S**tack of plates

BFS → **B**road → **Q**ueue at the bank

7. Informed Search

Informed search = a search strategy that uses **problem-specific knowledge** to find solutions **more efficiently**.

Why?

DFS and BFS explore blindly. In a maze, a human would naturally move **toward** the exit. Informed search gives the computer that same “sense of direction”.

Heuristic Function — $h(n)$

Think of it like this (real life):

You are walking to a tall tower you can see in the distance. You don’t know the exact streets, but you can **see** that the tower is “roughly that way, about 2 km”. That rough feeling is a **heuristic**. It is not exact (there may be a river in the way), but it helps you choose which street to try first.

Heuristic function $h(n)$ = a function that **estimates** how close node n is to the goal (the estimated cost from n to the goal).

- It is a **guess**, not the exact answer.
- It must be **quick** to calculate (otherwise it is useless).
- $h(\text{goal}) = 0 \rightarrow$ you are already there.

Manhattan Distance

For a grid (maze), the most common heuristic is the **Manhattan distance**: count how many squares you need to move **horizontally + vertically** to reach the goal, **ignoring the walls**.

$$h(n) = |x_n - x_{\text{goal}}| + |y_n - y_{\text{goal}}|$$

Example:

You are at (2, 3). Goal is at (6, 1).

$$h = |2 - 6| + |3 - 1| = 4 + 2 = \mathbf{6}.$$

Grid showing Manhattan distance of every square to the goal **G** (top-right):

5	4	3	2	1	G
6	5	4	3	2	1
7	6	5	4	3	2
8	7	6	5	4	3

start

It is called “Manhattan” because you move in straight blocks like the streets of New York’s Manhattan (no diagonal shortcuts).

Manhattan distance **ignores walls**. So a square with a small h may actually be in a dead end. That’s why it is only an *estimate*.

Simple:

A heuristic is a quick guess of “how far am I from the goal?”

1. Greedy Best-First Search (GBFS)

Think of it like this (real life):

A greedy person always takes the street that points **most directly** at the tower, without thinking. Sometimes that street ends at a river, and they have to go all the way around — a long walk.

Greedy best-first search = a search algorithm that expands the node that is **closest to the goal**, as estimated by a heuristic function $h(n)$.

Always remove the frontier node with the smallest $h(n)$

It is called **greedy** because it always grabs what looks best **right now**, without thinking about the total cost.

- **Good:** usually explores **far fewer** nodes than DFS/BFS — it heads straight toward the goal.
- **Bad: not optimal.** It ignores how far it has already travelled, so it can be tricked by a path that *looks* close but is actually long.

2. A* Search (“A-star”)

Think of it like this (real life):

A smart person thinks: “How far have I **already walked** on this route, **plus** how far does it **still look?**” They choose the route with the smallest **total**. Like a taxi app comparing full trip lengths, not only “which road points toward the destination”.

A* search = a search algorithm that expands the node with the **lowest value of $g(n) + h(n)$** .

$$f(n) = g(n) + h(n)$$

Symbol	Meaning
$g(n)$	Real cost already paid to reach n (steps taken so far). → the past
$h(n)$	Estimated cost from n to the goal (heuristic). → the future
$f(n)$	Estimated total cost of the best path that goes through n .

Why?

If a path has already become very long (g is big), A* will switch to another path that has a smaller total, even if it looks further from the goal. Greedy would not.

Example:

(from the lecture's maze) Two squares in the frontier:

Square X: took **11** steps to reach, Manhattan distance **6** → $f = 11 + 6 = 17$.

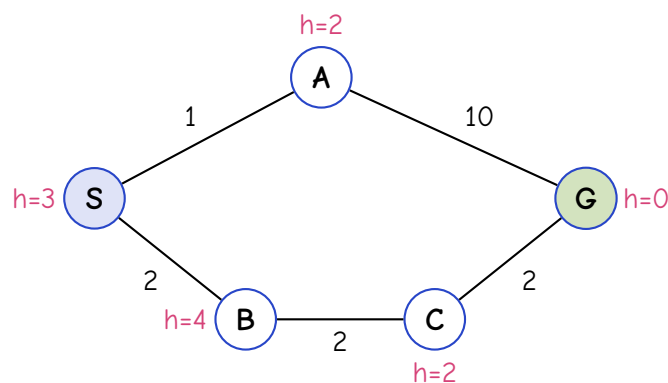
Square Y: took **15** steps to reach, Manhattan distance **4** → $f = 15 + 4 = 19$.

Greedy picks **Y** ($h = 4$ is smaller).

A* picks **X** ($f = 17$ is smaller) → X's route is expected to be cheaper overall.

Worked Example: Greedy vs A*

Task: go from **S** to **G**. Numbers on edges = step cost. Red numbers = heuristic h .



Two routes: $S \rightarrow A \rightarrow G = 1 + 10 = 11$ $S \rightarrow B \rightarrow C \rightarrow G = 2 + 2 + 2 = 6$ (optimal).

Greedy Best-First (uses only h)

Step	Remove	Frontier (with h)
1	S	A(2), B(4)
2	A (smallest $h = 2$)	B(4), G(0)
3	G (smallest $h = 0$)	Goal!

Greedy returns $S \rightarrow A \rightarrow G$ with cost **11** → **NOT optimal**. A looked close ($h=2$) but the road $A \rightarrow G$ was very expensive.

A* (uses $g + h$)

Step	Remove	Frontier (with $f = g + h$)
1	S ($0 + 3 = 3$)	A($1 + 2 = 3$), B($2 + 4 = 6$)
2	A ($f = 3$)	B(6), G($11 + 0 = 11$)
3	B ($f = 6$)	C($4 + 2 = 6$), G(11)
4	C ($f = 6$)	G($6 + 0 = 6$) (cheaper route to G replaces the old 11)
5	G ($f = 6$)	Goal!

A* returns $S \rightarrow B \rightarrow C \rightarrow G$ with cost **6** → **optimal**.

Notice step 2: A* saw G with cost 11 but did **not** stop, because B had a smaller f (6). A* only stops when the goal is **removed** from the frontier, not when it is first seen.

When is A* Optimal?

A* is guaranteed to find the optimal solution if the heuristic is:

1. Admissible — never overestimates

Admissible heuristic = $h(n)$ is **never more** than the true cost from n to the goal. $h(n) \leq$ true cost.

The heuristic can be **too low** (optimistic), but never **too high**.

Example:

- Manhattan distance in a grid maze is **admissible**: walls can only make the real path **longer**, never shorter.
- Straight-line distance on a road map is admissible: no road is shorter than a straight line.
- In the example above: true cost from A is 7 ($A \rightarrow S \rightarrow B \rightarrow C \rightarrow G$), $h(A) = 2 \leq 7$ ✓.

Why?

If h overestimates, A* might think the best path is expensive and skip it.

2. Consistent — never drops too fast

Consistent heuristic = for every node n and its successor n' with step cost c : $h(n) \leq h(n') + c$.

In words: moving one step (cost c) can reduce the estimate by **at most** c .

Example:

$S \rightarrow A$ has cost 1. $h(S) = 3$, $h(A) = 2$. Check: $3 \leq 2 + 1 = 3$ ✓
 $B \rightarrow C$ has cost 2. $h(B) = 4$, $h(C) = 2$. Check: $4 \leq 2 + 2 = 4$ ✓

Choosing a heuristic

- $h(n) = 0$ for every node $\rightarrow$ admissible but useless (A^* behaves like a plain cost-based search).
- The **closer** h is to the true cost (without going over), the **fewer** nodes A^* explores.

Simple:

A^* = real cost so far + guess to go. Best path guaranteed if the guess never overestimates.

Easy Memory Trick:

Greedy $\rightarrow$ only **h** (looks only ahead) $A^* \rightarrow \mathbf{g} + \mathbf{h}$ (looks back **and** ahead)

Admissible $\rightarrow$ "Always under (or equal)"

All Four Search Algorithms – Comparison

Algorithm	Type	Frontier removes	Optimal?	Speed
DFS	Uninformed	Newest (stack)	No	Depends on luck
BFS	Uninformed	Oldest (queue)	Yes (equal costs)	Slow, lots of memory
Greedy Best-First	Informed	Smallest $h(n)$	No	Usually fast
A^*	Informed	Smallest $g(n) + h(n)$	Yes (good h)	Fast + correct

8. Adversarial Search

Think of it like this (real life):

Finding your way in a maze is easy compared to playing chess with a clever friend. The maze never tries to stop you. Your friend **does**. Every time you plan a move, you must ask: "What will my friend do to spoil it?"

Adversarial search = search in a situation where an **opponent** (adversary) is trying to reach the **opposite** goal — to make you lose.

Why?

In a maze, the maze doesn't fight back. In a game, every time you move, the opponent replies with the move that is **worst for you**. So we must plan for the opponent too.

Example:

Tic-Tac-Toe, Chess, Checkers, Connect Four. (Two players, take turns, both can see the full board.)

Minimax — The Idea

Think of it like this (real life):

Imagine a scoreboard with a single number. You (X) want that number **as high as possible**. Your friend (O) wants it **as low as possible**. Before moving, you imagine: “If I do this, what is the best thing my friend can do back? Then what’s my best reply?” — and you keep imagining until the game ends. Then you choose the move that leaves you best off **even if your friend plays perfectly**.

The two players are named after what they want:

- **MAX (X)** → aims to **maximize** the score.
- **MIN (O)** → aims to **minimize** the score.

Every finished game gets a number (**utility**):

Result	Utility	Good for
X wins	+1	MAX
Draw (tie)	0	nobody
O wins	−1	MIN

Turning a game into numbers means the computer does not need to “understand” Tic-Tac-Toe — it just tries to push the number **up** (MAX) or **down** (MIN).

A Game as a Search Problem

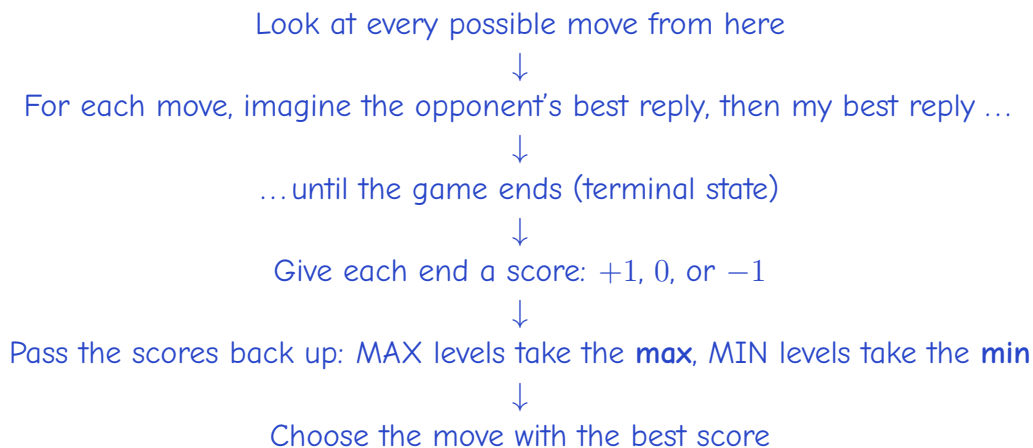
A game is described by these 6 parts:

Part	Meaning	Tic-Tac-Toe example
S_0	Initial state	Empty 3×3 board
PLAYER(s)	Which player moves in state s	Empty board → X. After X moves → O.
ACTIONS(s)	Legal moves in state s	All empty squares
RESULT(s, a)	State after action a in state s	The board with the new X or O placed
TERMINAL(s)	Is the game over in state s?	True if someone has 3 in a row or the board is full
UTILITY(s)	Final value of a terminal state	+1 / 0 / −1

How Minimax Chooses a Move

Given a state s:

- **MAX** picks the action a from $ACTIONS(s)$ that produces the **highest** value of:
 $MIN-VALUE(RESULT(s, a))$
 (“Which move is best for me, assuming O then plays perfectly?”)
- **MIN** picks the action a from $ACTIONS(s)$ that produces the **smallest** value of:
 $MAX-VALUE(RESULT(s, a))$
 (“Which move is worst for X, assuming X then plays perfectly?”)



Minimax Rules in Plain Words

To find the value of any board position, ask **whose turn** it is:

Situation	What to do
Game is over	The value is simply the final score (+1, 0 or −1). Nothing more to think about.
MAX's turn (X)	Look at every possible move. Find the value of each resulting board (imagining O's best reply). Take the biggest value.
MIN's turn (O)	Look at every possible move. Find the value of each resulting board (imagining X's best reply). Take the smallest value.

Key ideas behind these rules

- **The game over case** is where the thinking stops. Every imagined line of play eventually ends in a finished game with a real score.
- **MAX starts from “worst possible”** ($-\infty$): before looking at any move, MAX assumes nothing good. Each move it checks can only improve on that, so the best one is kept.
- **MIN starts from $+\infty$** for the same reason, in the opposite direction.
- **Players take turns in the imagination too:** to judge MAX's move, we think like MIN; to judge MIN's reply, we think like MAX, and so on. Thinking about a smaller version of the same question again and again is called **recursion**.

Tic-Tac-Toe Example

It is **X's turn** (MAX). Only 2 squares are empty (marked 1 and 2):

O	X	O
O	X	X
X	1	2

Option 1: X plays square 1.

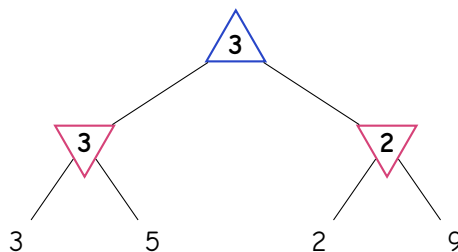
The middle column becomes X, X, X → **X wins** → value = +1.

Option 2: X plays square 2.

Now it is O's turn, and only square 1 is left. O plays it → board full, nobody has 3 in a row → **draw** → value = 0.

MAX chooses $\max(+1, 0) = +1 \rightarrow$ **play square 1**

Small Minimax Tree



($\triangle$ = MAX's turn, ∇ = MIN's turn. Bottom numbers = final scores.)

1. Left MIN node: MIN chooses $\min(3, 5) = 3$.
2. Right MIN node: MIN chooses $\min(2, 9) = 2$.
3. MAX root: MAX chooses $\max(3, 2) = 3 \rightarrow$ MAX plays the **left** move.

MAX does **not** go right just because 9 is the biggest number there. MIN would never let MAX get 9 — MIN would pick 2. Minimax always assumes the opponent plays their **best**.

Properties of Minimax

- **Optimal** against a perfect opponent: it gets the best result that can be guaranteed.
- **Very slow** for big games: it looks at **every** possible game until the end.

Simple:

Minimax = "I pick my best move, assuming you pick your best reply, and so on to the end."

Easy Memory Trick:

MAX → triangle points **up** → wants score **up** MIN → triangle points **down** → wants score **down**

9. Optimizing Minimax

The Problem: Far Too Many Games

Game	Number of possible games
Tic-Tac-Toe (all complete games)	255,168
Chess (after only 4 moves each, lower bound)	288,000,000,000
Chess (full game)	more than the atoms in the universe

A computer can check all 255,168 Tic-Tac-Toe games easily. For chess it is **impossible**. Two tricks help:

1. Alpha-Beta Pruning

Think of it like this (real life):

You are choosing a restaurant. Restaurant 1's *worst* dish is still good (rated 4/5). At Restaurant 2, the first dish you read about is terrible (rated 1/5), and you know you'll get the *worst* dish there. You don't need to read the rest of Restaurant 2's menu — it's already worse than Restaurant 1. Skipping the rest = **pruning**.

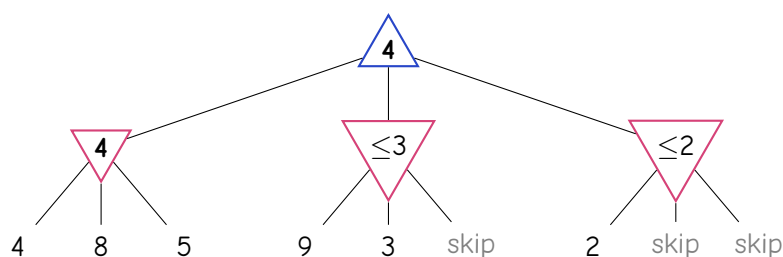
Pruning = cutting off (skipping) branches of the game tree that **cannot change** the final decision.

Alpha-beta pruning = Minimax that keeps track of the best scores found so far, and stops exploring a move as soon as it is clear that move is worse than one already found.

- α (**alpha**) = best value MAX can **guarantee** so far (starts at $-\infty$).
- β (**beta**) = best (lowest) value MIN can **guarantee** so far (starts at $+\infty$).
- If at any point $\alpha \geq \beta \rightarrow$ stop exploring this branch (prune).

Example

MAX has 3 possible moves. After each, MIN has 3 replies:



1. **Left MIN:** check all three $\rightarrow \min(4, 8, 5) = 4$.
So MAX can already **guarantee at least 4** ($\alpha = 4$).

2. **Middle MIN:** sees 9 (no help), then sees 3.
MIN will pick 3 **or something even lower** → this move is worth **at most 3**.
 $3 < 4$ → MAX will never choose this move → **skip the last child**.
3. **Right MIN:** first child is 2 → this move is worth at most 2.
 $2 < 4$ → **skip the other two children**.
4. MAX picks the left move with value 4.

We got the **same answer** as full Minimax, but skipped 3 of the 9 leaves. In big games, alpha-beta can skip **huge** parts of the tree.

Why?

It doesn't matter what the skipped values are. Even if one of them were 1000, MIN would never pick it — MIN already has something smaller.

Simple:

Stop checking a move as soon as you know it is worse than a move you already have.

2. Depth-Limited Minimax

Think of it like this (real life):

A chess player cannot imagine the whole game to the end. Instead they think "3–4 moves ahead", then look at the board and judge: "I have more pieces, my king is safe, so I'm probably winning." That judgement is the **evaluation function**.

Depth-limited Minimax = Minimax that only looks a **fixed number of moves ahead** (e.g. 10 moves), instead of all the way to the end of the game.

Look N moves ahead → Stop → Estimate who is winning

Problem: at that depth, the game is usually **not over**. There is no real UTILITY (+1/0/−1).

Solution: use an **evaluation function** to estimate the score.

Evaluation function = a function that **estimates the expected utility** of the game from a given state.

Example:

(Chess) Count material: Queen = 9, Rook = 5, Bishop = 3, Knight = 3, Pawn = 1.

Score = (my pieces' total) − (opponent's pieces' total).

I have 39, opponent has 30 → score = +9 → good for me.

Better evaluation functions also consider king safety, piece position, control of the centre, etc.

Example:

(Tic-Tac-Toe) Score = (lines X can still win) − (lines O can still win).

- **Good** evaluation function → strong AI player.
- **Bad** evaluation function → weak AI player, even with deep search.

Trade-off: **deeper** search = better moves but **slower**. Real chess programs combine depth-limited search + alpha-beta pruning + a strong evaluation function.

Simple:

Can't see the end of the game? Look a few moves ahead and **estimate** who is winning.

Easy Memory Trick:

Alpha-Beta → **skip** useless branches (same answer, faster)

Depth-Limited → **stop early** and estimate (faster, but an approximation)

10. Quick Revision

Remember

- **Agent** = perceives + acts. **State** = one configuration.
- **Search problem** = Initial state, Actions, Transition model, Goal test, Path cost.
- $ACTIONS(s)$ = legal moves. $RESULT(s, a)$ = new state after move.
- **State space** = all reachable states (a graph).
- **Solution** = any path to goal. **Optimal** = lowest path cost.
- **Node** = state + parent + action + path cost. Parent lets us rebuild the path.
- **Frontier** = to-do list. **Explored set** = already visited (prevents loops).
- **DFS** = deepest first, Stack (LIFO), not optimal, low memory.
- **BFS** = shallowest first, Queue (FIFO), optimal for equal costs, high memory.
- **Uninformed** = no hints. **Informed** = uses a heuristic $h(n)$.
- **Manhattan distance** = $|x_1 - x_2| + |y_1 - y_2|$, ignores walls.
- **Greedy** = smallest $h(n)$; fast, not optimal.
- **A*** = smallest $g(n) + h(n)$; optimal if h is **admissible** ($\leq$ true cost) and **consistent** ($h(n) \leq h(n') + c$).
- **Minimax**: MAX (X) maximizes, MIN (O) minimizes. Utility +1, 0, -1.
- Game = S_0 , PLAYER, ACTIONS, RESULT, TERMINAL, UTILITY.
- **Alpha-Beta** = prune branches that can't change the answer.
- **Depth-Limited** = look N moves ahead + **evaluation function**.

11. Exam Questions

Q1. Define a search problem and list its components with an example.

Ans: A search problem is one where an agent must find a sequence of actions from an initial state to a goal state. Components: initial state, actions $ACTIONS(s)$, transition model $RESULT(s, a)$, goal test, and path cost function. Example (maze): initial state = start square;

actions = move up/down/left/right; transition model = the neighbouring square; goal test = “on exit square?”; path cost = 1 per step.

Q2. What is a node? Why does it store its parent?

Ans: A node is a data structure that stores a state, its parent node, the action used to reach it, and the path cost. The parent is stored so that when the goal is found, we can follow the parents back to the start and rebuild the full path (then reverse it).

Q3. Why do we need an explored set?

Ans: Without it, the search can revisit the same states again and again (e.g. $A \rightarrow B \rightarrow A \rightarrow B \dots$) and loop forever. The explored set remembers visited states so they are never added to the frontier again.

Q4. Why does DFS use a stack and BFS use a queue?

Ans: A stack returns the **newest** node (LIFO), so the search continues from the last node found, going deeper — that is DFS. A queue returns the **oldest** node (FIFO), so all nodes at one depth are explored before the next depth — that is BFS.

Q5. Which of DFS and BFS is optimal? Why?

Ans: BFS, when all steps cost the same. It explores level by level, so the first time it reaches the goal is by the path with the fewest steps. DFS returns the first path it happens to find, which may be long.

Q6. Why is A* better than Greedy Best-First Search?

Ans: Greedy uses only $h(n)$, so it may follow a path that looks close but is actually expensive (in the worked example it returned cost 11 instead of 6). A* uses $g(n) + h(n)$, so it also considers the cost already paid. With an admissible and consistent heuristic, A* always finds the optimal path.

Q7. What is an admissible heuristic? Give an example.

Ans: A heuristic that never overestimates the true cost to the goal ($h(n) \leq \text{true cost}$). Manhattan distance in a grid maze is admissible because walls can only make the real path longer.

Q8. Explain the Minimax algorithm.

Ans: Minimax is for two-player games. MAX tries to maximize the score, MIN tries to minimize it. The algorithm explores all moves to the end of the game, gives each ending a utility (+1, 0, -1), and passes values upward: MAX nodes take the maximum of their children, MIN nodes the minimum. MAX then plays the move with the highest value, assuming MIN plays perfectly.

Q9. Why can't we run full Minimax on chess? How do we fix it?

Ans: There are too many possible games (over 288 billion after just 4 moves each). We use **alpha-beta pruning** to skip branches that can't change the decision, and **depth-limited Minimax** which looks only a few moves ahead and uses an **evaluation function** to estimate the score.

Practice Questions

1. Define agent, state, action, transition model, state space, goal test, and path cost.
2. What is the difference between a solution and an optimal solution? Give an example.
3. Write the 5 components of a search problem for the 15-puzzle.
4. What 4 things does a node store? What is the difference between a node and a state?
5. Write the revised search algorithm (with explored set).

6. Trace DFS and BFS on a small graph. Show the frontier and explored set at each step.
7. Compare DFS and BFS (frontier, completeness, optimality, memory).
8. What is the difference between uninformed and informed search?
9. What is a heuristic? Calculate the Manhattan distance from (1, 1) to (4, 5). (Ans: 7)
10. Trace Greedy Best-First Search and A* on a weighted graph. Which one is optimal?
11. Define admissible and consistent heuristics. Check if a given heuristic is admissible.
12. List the 6 functions that describe a game and explain each for Tic-Tac-Toe.
13. Explain the Minimax rules in your own words. Why does MAX start with $v = -\infty$?
14. Solve a small Minimax tree by hand.
15. Explain Alpha-Beta Pruning with an example. Does it change the final answer?
16. What is Depth-Limited Minimax? What is an evaluation function? Give a chess example.